

COLLEGE OF COMPUTING AND INFORMATICS

DEPARTMENT OF SOFTWARE ENGINEERING

COURSE: SOFTWARE DESIGN AND ARCHTICTURE

PROJECT TITLE : SMART PERSONAL FINANCE DASHBOARD

Name ID
1. Mekuanint Maru 1600866
2. Kalkidan Getnet 1600721
3. Yosef Sewnet 1601374

Submitted to: ms. Destaye

Date of submission:30/08/2018 E

Table of Contents

1. INTRODUCTION 1
    1.1 Purpose 1
    1.2 Scope 2
    1.3 Overview 3
    1.4 Intended Users 4
    1.5 Reference Material 4
    1.6 Definitions and Acronyms 5
2. SYSTEM OVERVIEW AND CONSTRAINTS 6
    2.1 System Overview 6
    2.2 Constraints 8
3. QUALITY ATTRIBUTE SCENARIOS AND TACTIC SELECTION 10
    3.1 Quality Attribute Scenarios 10
    3.2 Tactic Selection 11
4. SYSTEM ARCHITECTURE 12
    4.1 Architectural Design 12
    4.2 Module View 13
    4.3 Component and Connector View 15
    4.4 Allocation View 17
    4.5 Design Rationale 18
5. DATA DESIGN 19
    5.1 Data Description 19
    5.2 Data Dictionary 20
6. COMPONENT DESIGN 23
    6.1 Use Case Diagram 23
    6.2 Deployment Diagram: 25
7. REQUIREMENTS MATRIX 288

1. INTRODUCTION

1.1 Purpose

The primary purpose of this Software Design Document (SDD) is to provide a complete and formal technical description of the **Smart Personal Finance Dashboard**. This document acts as a "blueprint" for the entire development lifecycle. Its specific objectives are:

- **Design Communication:** To translate high-level business requirements into technical specifications that can be understood by developers, testers, and project supervisors.
- **Architectural Clarity:** To define how different software components (React frontend, Spring Boot backend, and MySQL database) interact with each other to ensure a cohesive system.
- **Quality Assurance:** To establish a baseline for evaluating the system's performance, security, and scalability before the actual coding and deployment phases.
- **Maintenance Roadmap:** To serve as a historical record for future maintenance or system upgrades, allowing new developers to understand the logic and structure of the application quickly.
- **Risk Mitigation:** To identify potential design bottlenecks or security vulnerabilities early in the process, reducing the cost and effort of fixing issues during the production phase.

1.2 Scope

The scope of the Smart Personal Finance Dashboard project encompasses the design, development, and implementation of a centralized web platform for individual financial management.

The system's functional scope includes:

- **User Management:** A secure authentication system using JWT (JSON Web Tokens) that allows users to register, log in, and manage their personal profiles.
- **Transaction Processing:** A module for users to record income and expenses with detailed metadata such as dates, descriptions, and amounts.
- **Budgeting Engine:** A logic-based component that allows users to set financial boundaries for various categories (e.g., Housing, Groceries, Entertainment) and monitors spending against these limits.
- **Advanced Analytics:** An interactive dashboard that uses data visualization tools to present spending trends, category distributions, and financial health scores.
- **Data Persistence:** A robust relational database schema designed to store years of financial data without compromising retrieval speed.

The scope is technically bounded by:

- **Frontend:** Built using the React.js framework for a responsive, single-page application (SPA) experience.
- **Backend:** Developed with the Java Spring Boot framework, following a RESTful API architecture.
- **Database:** Utilizing MySQL for structured data storage and management.

Out of Scope (What the system will NOT do):

- The current version will not support multi-currency conversion (it uses a single base currency).
- It does not include direct integration with cryptocurrency wallets or stock market trading platforms in this release.
- Automated bank synchronization via scraping is excluded; data entry is via manual logging or CSV import.

1.3 Overview

This document is organized into seven major sections:

- Section 2 provides the system context and constraints.
- Section 3 details the quality attribute scenarios and tactics.
- Section 4 illustrates the system architecture and various views.
- Section 5 and 6 focus on data and component-level design.
- Section 7 provides the requirements traceability matrix.

1.4 Intended Users

This document is intended for the following audience:

- **Software Developers:** To guide the implementation of the backend APIs and frontend components.
- **System Architects:** To review the structural integrity and design patterns used.
- **Project Supervisors/Evaluators:** To assess the technical depth and design quality of the software engineering project.
- **Database Administrators (DBAs):** To understand the schema requirements and entity relationships.

1.5 Reference Material

- Software Requirements Specification (SRS) for Smart Personal Finance Dashboard.
    - *IEEE Standard for Information Technology—Systems Design—Software Design Descriptions (IEEE Std 1016-2009).*
    - *Spring Boot Official Documentation:* https://spring.io/projects/spring-boot
    - *React.js Official Documentation:* https://react.dev/
    - *UML 2.5 State Machine Diagrams and Notations.*

1.6 Definitions and Acronyms

Term/Acronym	Definition
API	Application Programming Interface; a set of protocols for building software applications.
ERD	Entity-Relationship Diagram; a snapshot of how data is stored in the database.
JWT	JSON Web Token; a compact, URL-safe means of representing claims to be transferred between two parties.
MVC	Model-View-Controller; a design pattern used to decouple user interface, data, and application logic.
REST	Representational State Transfer; an architectural style for providing standards between computer systems on the web.
UI/UX	User Interface and User Experience.
JPA	Java Persistence API; a specification for accessing, managing, and persisting data between Java objects and relational databases.

2. SYSTEM OVERVIEW AND CONSTRAINTS

2.1 System Overview

The Smart Personal Finance Dashboard is a web application that centralizes a user's financial life. It integrates transaction logging, categorical budgeting, and analytical charts into a single interface. The system is built using a Spring Boot backend and a React.js frontend to provide a responsive and secure user experience.

The **Smart Personal Finance Dashboard** is an intelligent financial management tool designed to help users take control of their economic life. In an era of digital transactions, tracking every expense manually is difficult. This system automates the process by allowing users to categorize expenses, set monthly limits (budgets), and receive real-time feedback on their financial health.

The system architecture is designed for high security and performance, ensuring that sensitive financial data is protected while providing a seamless, interactive user experience through a modern web dashboard.

The system is built on several core functional pillars:

- **Financial Tracking:** Users can log daily transactions, specifying the amount, date, and description. The system automatically maintains a history of all financial activities.
- **Budget Management:** A proactive feature that allows users to set monthly or weekly spending limits for different categories (e.g., Food, Transport, Rent). The system tracks remaining balances and provides visual warnings when limits are approached.
- **Categorization & Organization:** Every transaction is linked to a specific category, enabling the system to organize data into meaningful structures for better decision-making.
- **Data Visualization:** Leveraging modern charting libraries, the dashboard transforms raw transaction data into interactive visual reports, such as pie charts for expense distribution and line graphs for monthly spending trends.
- **Security & Accessibility:** Built with a secure Spring Boot backend and a responsive React.js frontend, the system ensures that sensitive financial data is protected via encrypted authentication while remaining accessible from any web-enabled device.

By integrating these features into a single, cohesive interface, the Smart Personal Finance Dashboard transitions personal finance management from a tedious manual task into an automated, insightful, and user-friendly experience.

2.2 Constraints

The development and deployment of the **Smart Personal Finance Dashboard** are governed by several technical, operational, and design constraints that ensure the system is reliable and manageable:

- **Technical Stack Constraints:**
  - **Backend:** The system must be developed using **Advanced Java (Spring Boot framework)** to ensure enterprise-grade security and modularity.
  - **Frontend:** The user interface must be a Single Page Application (SPA) built with **React.js** to provide a seamless and responsive user experience.
  - **Database:** Only **MySQL** relational database is to be used for data persistence to ensure data integrity and support for complex financial queries.
- **Security Constraints:**
  - **Authentication:** The system is constrained to use **JWT (JSON Web Tokens)** for stateless authentication to maintain high security across API calls.
  - **Data Privacy:** Sensitive user data, especially passwords, must be encrypted using strong hashing algorithms (like **BCrypt**) before being stored in the database.
  - **Encryption:** All communication between the frontend and backend must be over **HTTPS** to prevent man-in-the-middle attacks.

- **Operational & Performance Constraints:**
  - **Response Time:** The system must be optimized so that data-heavy analytics (like generating spending graphs) load in less than **2 seconds** under normal network conditions.
  - **Availability:** The application is designed to be accessible **24/7**, requiring a hosting environment with high uptime (e.g., AWS or similar cloud providers).
  - **Browser Compatibility:** The dashboard must be fully functional on modern web browsers including **Google Chrome, Mozilla Firefox, and Safari**.

- **Design & Development Constraints:**
  - **UML Standards:** All architectural designs and diagrams must follow the **UML 2.5** (Unified Modeling Language) notation for consistency in documentation.
  - **Budgetary Constraints:** The project should prioritize open-source libraries and tools (like Spring Initializr, React, and MySQL Community Server) to minimize licensing costs.
  - **Timeline:** The initial version of the system, including core CRUD operations and budgeting logic, must be completed within the academic project deadline.

❖ The following constraints apply to the system's design and implementation:

- **Technology Stack:** The backend must be implemented using Advanced Java (Spring Boot).
- **Schedule:** The project must be completed and deployed by the end of the 2026 academic term.
- **Team Structure:** The project is a solo development effort, requiring efficient modular design.
- **Security:** Financial data must be encrypted to comply with privacy standards.

3. QUALITY ATTRIBUTE SCENARIOS AND TACTIC SELECTION

3.1 Quality Attribute Scenarios

1. **Security:** An unauthorized user attempts to access private financial data; the system must block access and log the attempt.
2. **Performance:** A user requests a year-to-date spending report; the system must generate and display the graph in under 2 seconds.
3. **Availability:** During a server failure, the system must recover within 5 minutes without losing transaction data.
4. **Modifiability:** A developer needs to add a new "Crypto-wallet" tracking feature; the system architecture should allow this without altering existing core logic.
5. **Usability:** A first-time user attempts to log a transaction; the system must provide a clear, intuitive form that can be completed in under 30 seconds.

3.2 Tactic Selection

Quality Attribute	Design Concern	Tactic Selected	Rationale
Security	Authentication	JWT & Spring Security	To ensure stateless and secure API communication.
Performance	Latency	Database Indexing	To speed up query execution for large transaction histories.
Availability	Fault Recovery	Spare/Redundancy	To maintain service during unexpected database outages.

3.2 table tactic selection

4. SYSTEM ARCHITECTURE

4.1 Architectural Design

The system is partitioned into high-level subsystems to ensure separation of concerns:

- **Client Subsystem (Frontend):** Responsible for UI rendering and state management.
- **API Gateway:** Manages routing and security filtering.
- **Finance Service (Backend):** Handles all core business logic and calculations.

Database Subsystem: Ensures data persistence and integrit

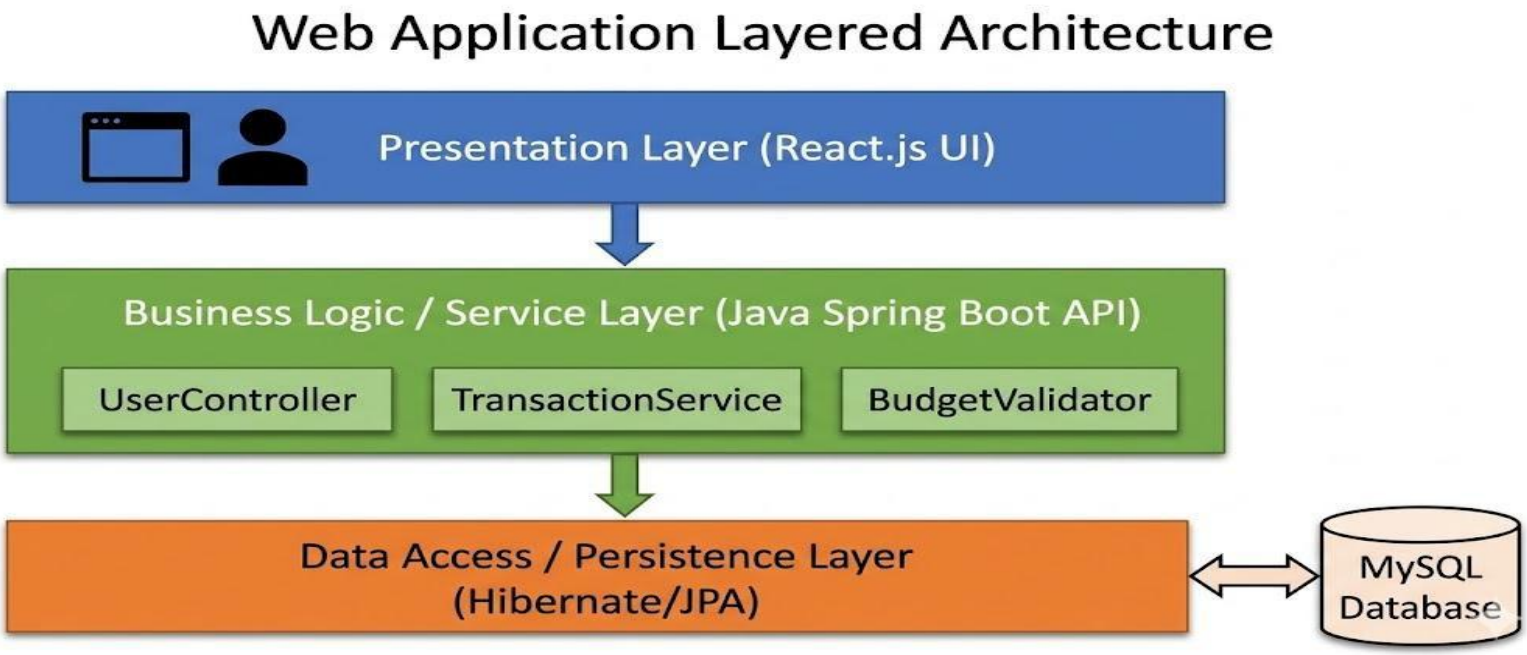


Figure 4.1: Three-Tier Layered Architecture Diagram

4.2 Module View

The system uses an **Object-Oriented Description**:

- **User Module:** Manages authentication and user profile data.
- **Transaction Module:** Handles CRUD operations for income and expense records.
- **Budget Module:** Tracks spending against user-defined limits.

The Class Diagram traces the static structure of the Java source code, representing the key entities, their attributes, methods, and relationships.

- **User Class:** Stores core user information (userId, username, email, passwordHash) and handles authentication and profile management.
- **Account Class:** Represents a financial account (accountId, type, balance), linking a user to their funds.
- **Transaction Class:** The core transactional entity (transactionId, amount, date, description). It has a composition relationship with the User and is categorized by a specific Category.
- **Category Class:** Defines expense or income categories (categoryId, name). It helps organize transactions for budgeting.
- **Budget Class:** Tracks spending limits set for specific categories over a set period (budget Id, limit, remaining Amount). It references a Category.

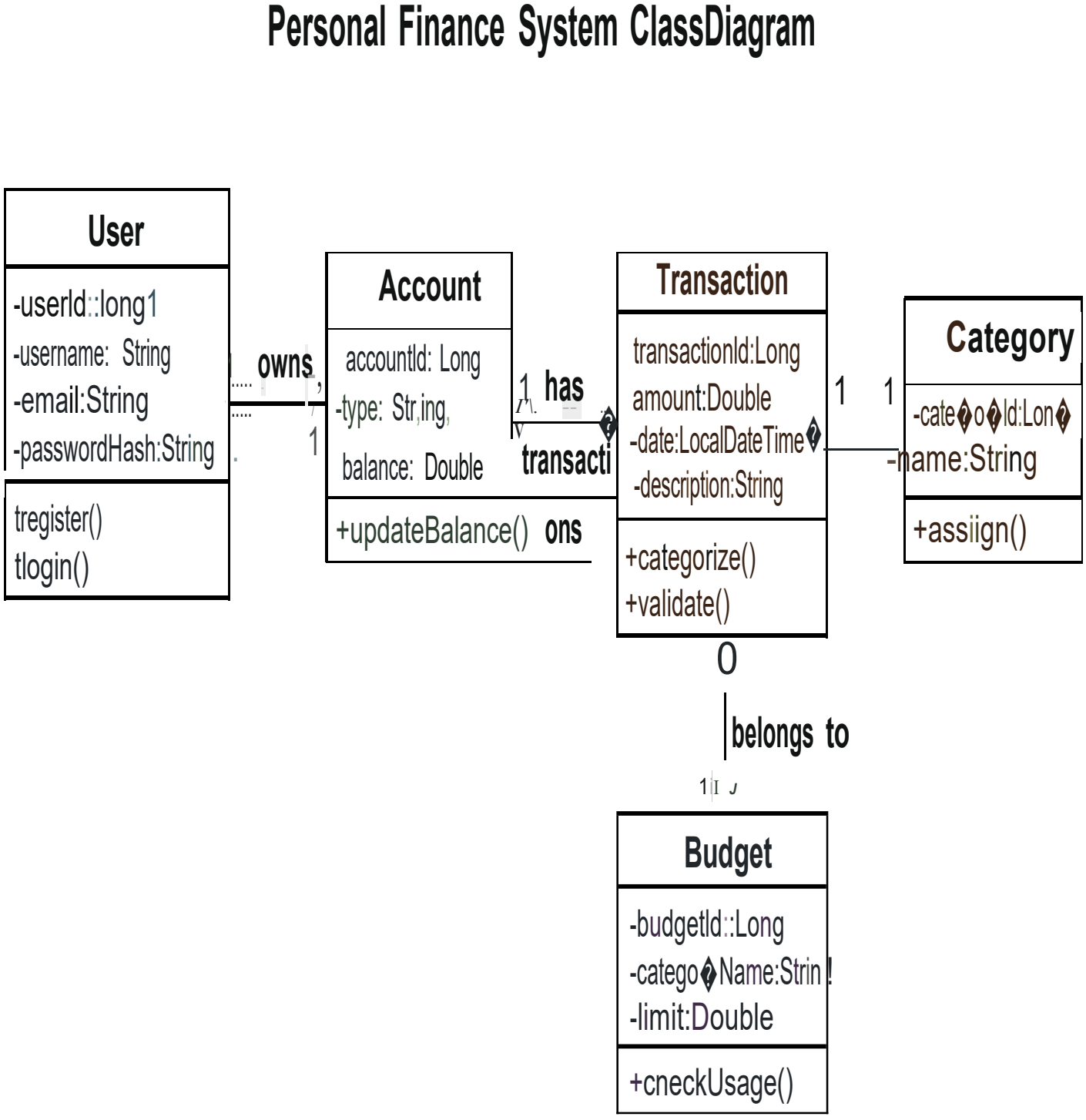


Figure 4.2 Module View (Class Diagram)

4.3 Component and Connector View

Runtime components include the **Spring Container** and the **MySQL Engine**. Connectors are implemented as **RESTful endpoints** over HTTPS to ensure secure data transfer between the browser and the server.

❖ Key Components:

- **Client Browser (React UI):** Responsible for presenting the user interface and handling user interactions.
- **API Gateway:** Acts as the single point of entry, routing requests to appropriate backend services.
- **Finance Service (Spring Boot):** The central computational component, managing business logic and connecting to external data.
- **MySQL Database:** Provides persistent data storage for users, transactions, and budget data.
- **External Bank API:** An external system that provides synchronized bank transaction data.

❖ Connectors:

- ✓ **HTTPS (REST API):** Secure connection facilitating data exchange between the browser and the system.
- ✓ **Internal HTTP:** Routes requests from the API Gateway to the specific services.
- ✓ **JDBC:** For interaction with the MySQL database engine.
- ✓ **Encrypted Connector:** Ensures secure connection to external financial APIs.

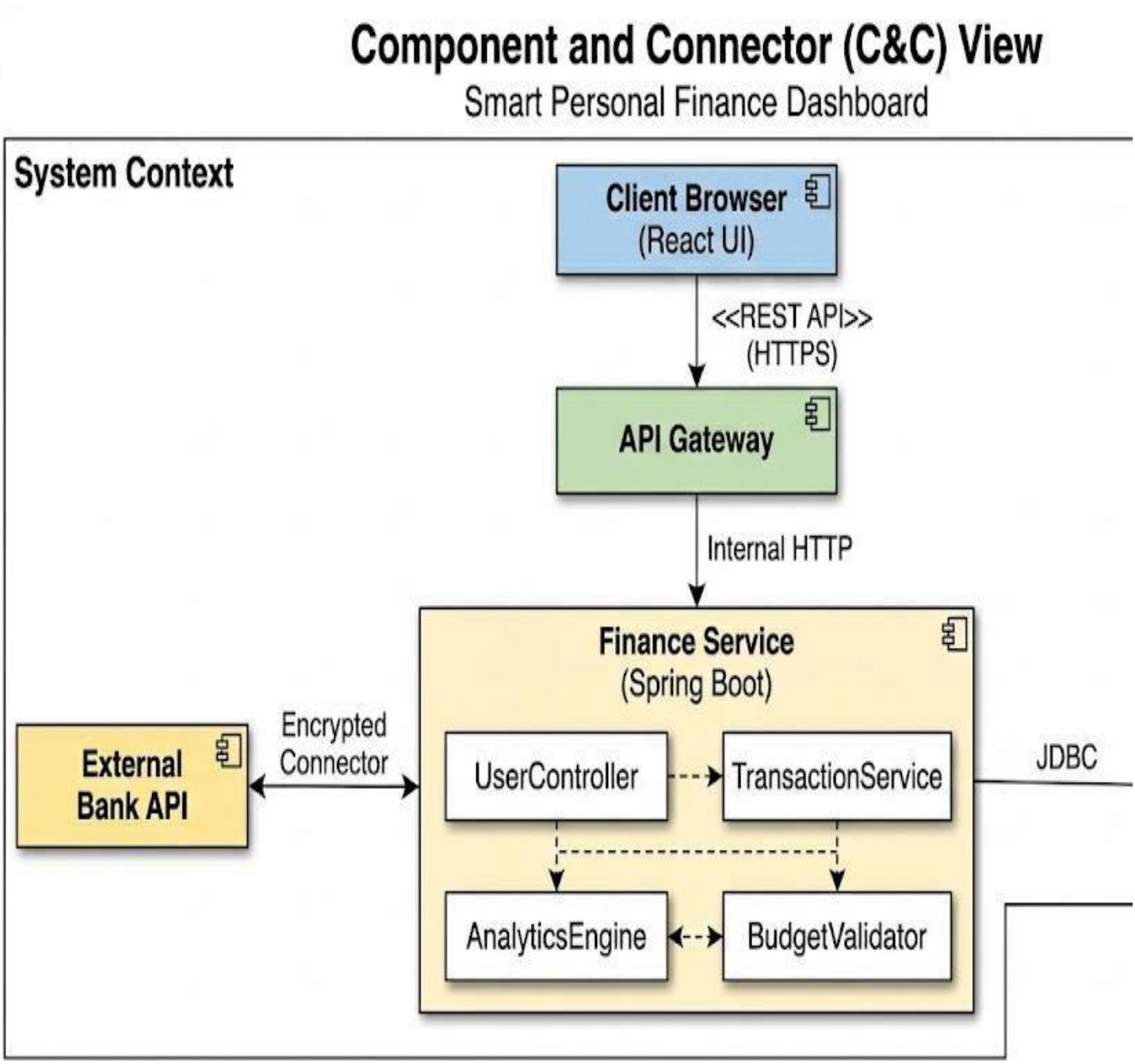


Figure 4.3 UML (Unified Modeling Language) Component Diagram notation"

4.4 Allocation View

The Allocation View describes how the software elements are mapped to non-software resources, such as hardware, network infrastructure, and development teams. it bridges the gap between the logical software design and the physical environment.

- **Deployment Mapping:** It defines how the Spring Boot backend and React frontend are hosted on cloud servers (e.g., AWS EC2) and how the MySQL database is allocated to a managed data node (e.g., AWS RDS).
- **Work Assignment:** It manages the distribution of tasks among the development team, ensuring that frontend developers, backend engineers, and database administrators work on their respective modules concurrently.
- **Network Allocation:** It specifies the bandwidth and communication protocols (like HTTPS and JDBC) assigned for data transfer between the client browser and the server.

4.5 Design Rationale

The Design Rationale provides the reasoning and justification behind the key architectural decisions made during the development of the Smart Personal Finance Dashboard. It explains "why" specific technologies and patterns were chosen over alternatives.

- **Choice of Layered Architecture:** A 3-tier layered architecture was selected to ensure a strict separation of concerns. This allows for independent updates to the UI (React) without affecting the core business logic (Spring Boot).
- **Choice of Spring Boot (Java):** Spring Boot was chosen for its robust security features (Spring Security), excellent support for RESTful APIs, and the reliability of the Java ecosystem in handling financial data.
- **Choice of React.js:** React was selected due to its component-based architecture and Virtual DOM, which provide a high-speed, responsive user experience essential for real-time financial tracking.
- **Choice of MySQL:** As a relational database, MySQL provides ACID compliance (Atomicity, Consistency, Isolation, Durability), which is a non-negotiable requirement for maintaining accurate and consistent financial transaction records.
- **Stateless Authentication (JWT):** Using JSON Web Tokens (JWT) allows the system to remain scalable and secure, as the server does not need to store session data, making it easier to handle multiple users simultaneously5

5. DATA DESIGN

5.1 Data Description

The Data Description provides a detailed overview of the information entities managed by the Smart Personal Finance Dashboard. It defines the nature of the data, its format, and its significance within the financial management lifecycle. The system primarily handles structured relational data to ensure high consistency and accuracy.

The core data entities include:

- **User Profile Data:** This consists of sensitive and non-sensitive information required for identification and security. It includes unique user identifiers, full names, encrypted passwords (using BCrypt), and email addresses used for account recovery and notifications.
- **Transaction Data:** This is the most dynamic data element in the system. It captures the details of every financial movement, including the transaction type (Income or Expense), the specific amount (decimal format), the timestamp of the occurrence, and a descriptive memo for user reference.
- **Budgetary Data:** This data represents the financial constraints set by the user. It includes categorical spending limits, start and end dates for budget cycles, and calculated fields that track "remaining balance" by comparing current transactions against the set limit.
- **Categorization Data:** A set of metadata used to organize transactions. This includes predefined categories (like Utilities, Rent, or Salary) and user-defined custom categories, each linked to specific icons or color codes for visual dashboard representation.

- **Analytical Data:** Derived data generated by the system's business logic. This includes aggregated monthly totals, percentage distributions of spending, and historical trend data used to populate interactive charts and reports.

Data Integrity and Persistence: All data described above is stored in a **MySQL relational database**. To maintain data integrity, the system enforces strict data types, mandatory fields (NOT NULL constraints), and referential integrity through primary and foreign key relationships.

5.2 Data Dictionary

- **Users Table:** Stores persistent user profile information with **userId** as the Primary Key.
- **Transactions Table:** Stores transaction records. It uses **transactionId** as its Primary Key and **userId** and **categoryId** as Foreign Keys to link back to the user and their categories.
- **Categories Table:** Defines available transaction categories with **categoryId** as the Primary Key. It has a one-to-many relationship with the Transactions table.

Data Dictionary

Object	Attribute	Type	Description
Transaction	amount	Double	The monetary value of the entry.
category	category	String	The classification (e.g., Food, Health).
User	password	String	Encrypted credentials for login.

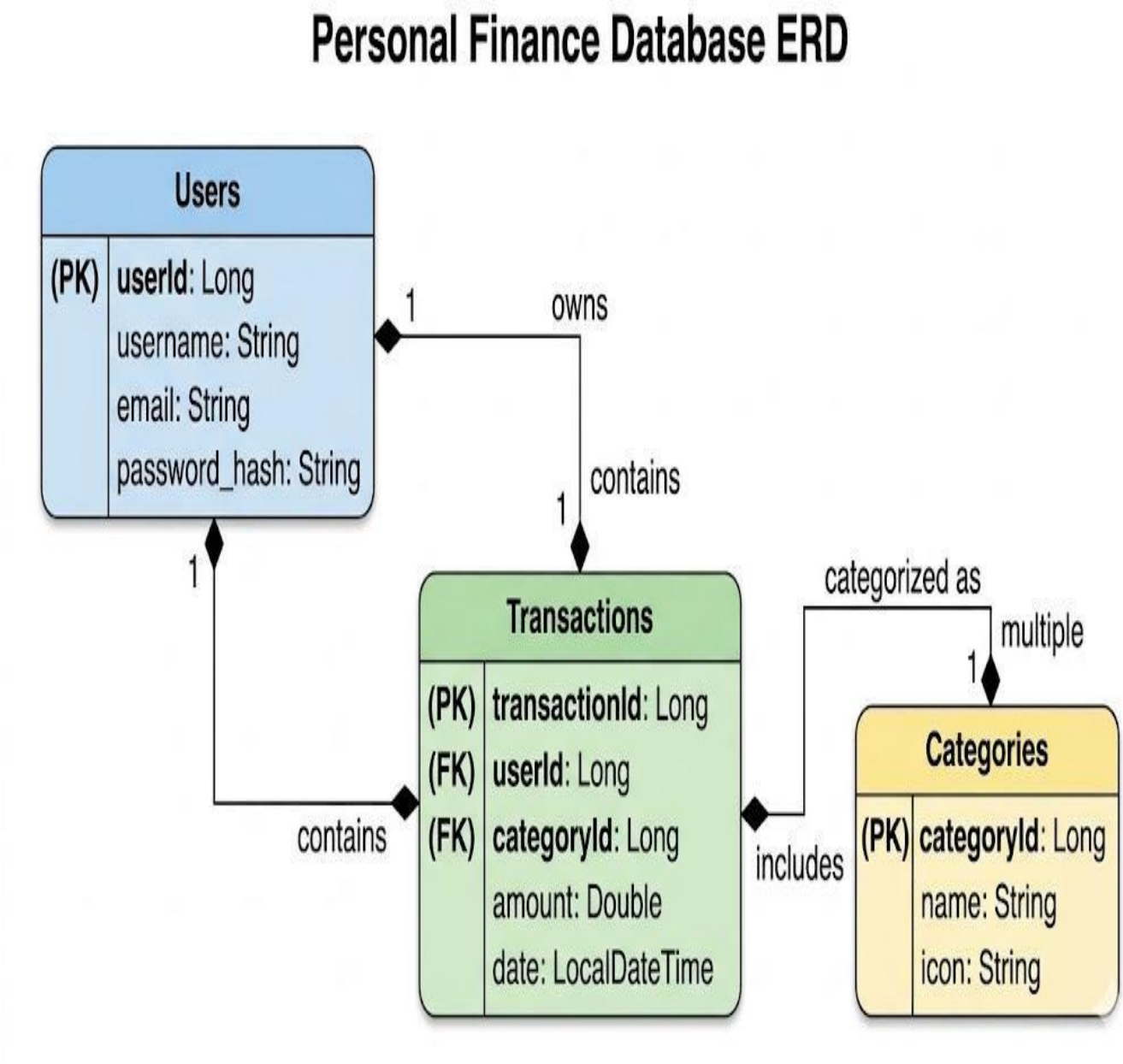


Figure 5.2: Database Entity-Relationship Diagram (ERD)

6. COMPONENT DESIGN

6.1 Use Case Diagram

The Use Case Diagram defines the system's functional requirements from the perspective of external actors, illustrating how they interact with specific use cases to achieve their goals.

- **Actors:**
  - **User:** Represents a standard application user.
  - **Authentication System:** A system-level actor involved in secure login.
- **Key Use Cases:**
  - **Register / Login:** Secure access control with extending "Manage User Profile."
  - **Manage Income & Expenses:** Core functionality for adding and categorizing transactions.
  - **Set Budgets & Financial Goals:** Allows users to define financial targets.
  - **View Financial Analytics & Charts:** Visual representation of spending habits.
  - **Generate Reports:** Export data for offline review.

This diagram shows the primary actor (User) interacting with major use cases like "Add Expense," "Analyze Monthly Spending," and "Set Savings Goal".

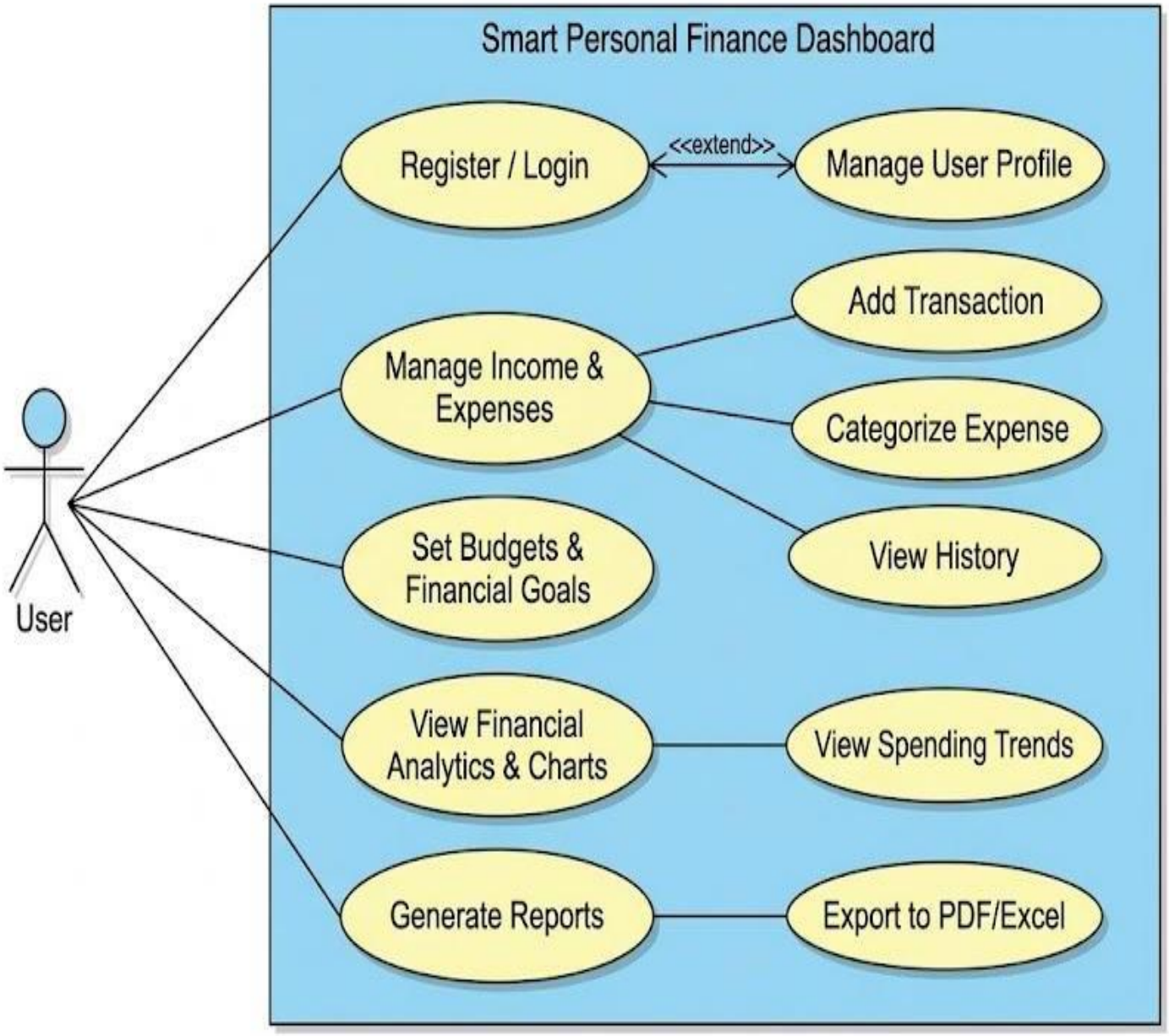


Figure 6.1: UML(Unified Modeling Language) Use Case Diagram for Personal Finance Dashboard

6.2 Deployment Diagram

The Deployment Diagram illustrates the physical configuration of the system's runtime software components and the hardware nodes they reside on. It provides a high-level view of how the application is distributed across the network infrastructure.

The software components are deployed across two primary nodes:

- **Web Server:** Hosts the Spring Boot application and React build files.
- **Database Server:** Runs the MySQL instance for data storage.

Key Infrastructure Components:

- **Client Device:** Represents the user's local machine (laptop, desktop, or mobile device). It hosts the **Web Browser**, which runs the React.js Presentation Layer. The connection to the server is established via secure **HTTPS** protocols.

- **Application Server (AWS EC2 Instance):** A cloud-based virtual server that hosts the backend environment. It runs the **Java Virtual Machine (JVM)** and the **Spring Boot Web Server**, which processes the business logic and manages REST API requests.

- **Database Server (AWS RDS Instance):** A dedicated, managed database service hosting the **MySQL Database**. The application server communicates with this node using **JDBC (Java Database Connectivity)** to perform persistent data operations.

- **External Bank API (Cloud-to-Cloud):** Represents third-party financial services that the system integrates with for real-time account synchronization via encrypted cloud connectors.

Network Protocols:

- **HTTPS:** Ensures secure data transmission between the client and the application server.
- **JDBC/JPA:** Facilitates communication between the service layer and the relational database.

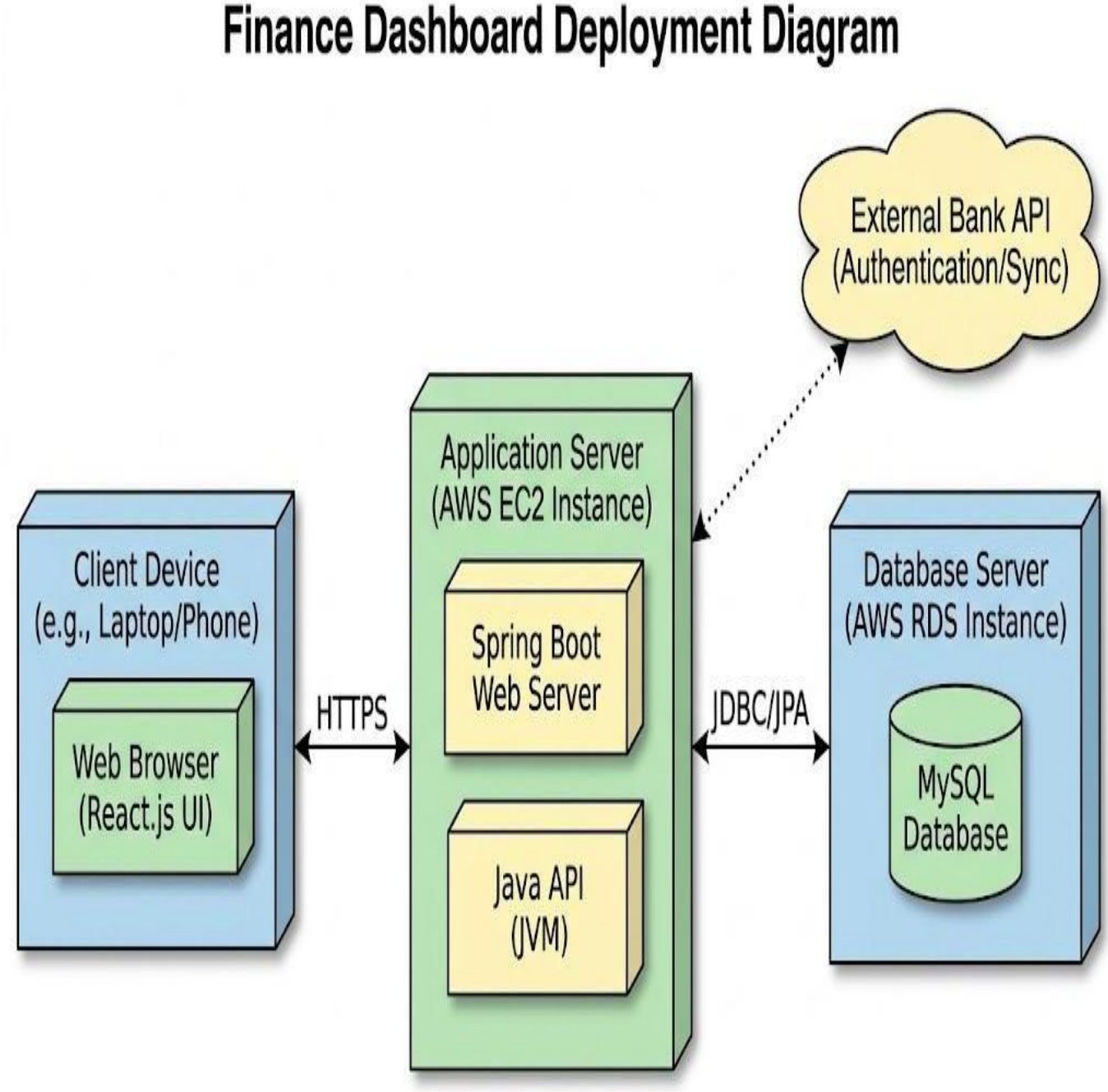


Figure 6.2: UML((Unified Modeling Language) Deployment Diagram showing Physical Nodes

7. REQUIREMENTS MATRIX

Requirement ID	Name	Description	System Component (Satisfying Module)
FR-1	User Registration	Allows new users to create an account with email and password.	**User Management Module** (Auth Controller)
FR-2	Secure Login	Provides JWT-based authentication for existing users.	**Security Service** (Spring Security)
FR-3	Add Income	Users can record salary or other earnings with date and source.	**Transaction Module** (Income Controller)
FR-4	Add Expense	Users can record spending and assign it to a category (e.g., Food).	**Transaction Module** (Expense Controller)
FR-5	Budget Limit	Users can set a maximum	**Budgeting Module** (Limit Validator)

Requirement ID	Name	Description	System Component (Satisfying Module)
		spending limit for a specific category.	
FR-6	Visual Dashboard	Displays spending trends using Pie-charts and Line-graphs.	**Analytics Module** (Dashboard Service)
FR-7	Data Export	Allows users to download their financial history as a PDF or Excel.	**Reporting Module** (Export Service)
NFR-1	Data Privacy	All sensitive financial data must be encrypted in the database.	**Database Layer** (Hibernate/Encryption)
NFR-2	Low Latency	Dashboard components must load within 2 seconds.	**Backend API** (Spring Boot Optimization)
NFR-3	Audit Trail	System logs all failed login attempts for	**Logging Subsystem** (Logback/SLF4J)

Requirement ID	Name	Description	System Component (Satisfying Module)
		security auditing.	
NFR-4	Accuracy	All financial calculations must be precise to two decimal places.	**Business Logic Layer** (Java Math API)
NFR-5	Reliability	The system must maintain 99.9% uptime during the academic year.	**Deployment Node** (AWS/Cloud Server)